dlprim · catalog of operators

Every method in catalog.py, grouped. A group is a base class whose `__call__` says what all its methods take and give. A method is a frozen dataclass that subclasses its group with the type parameters filled in: its fields are its settings, and calling it runs it.

Ops (expressions) take values and give values: a batch, a tree, a model as a value. They change nothing. **Commands** (statements) make new state, change it in place (the argument marked !, returning None), or read a whole dataset. State has a history, one version at a time, so a command is always a line of its own, never inside an expression. A sequence is a value (a 1-D tensor); a history is not.

R is what else a method takes (nothing by default): BestOfN(ReduceOp[Path, Path, [Value]]) takes a Path and a Value and gives a Path. Pyright checks every method's signature against its group's.

192 methods in 22 groups (24 base classes) · 130 ops, 62 commands (22 change their argument) · 165 model-agnostic, 27 model-dependent

The groups

group (base class)	what its methods take and give	methods
EXPRESSIONS · ops		
`ConvertOp[A, B, **R = []]`	`(x: A, /, *R) -> B`	8
`ElementwiseOp[A, B, **R = []]`	`(x: A, /, *R) -> B`	26
`LayerOp[A, B]`	`(x: A, /) -> B`	19
`ReduceOp[C, A, **R = []]`	`(x: C, /, *R) -> A`	4
`ScoreOp[P, O = Tensor, **R = []]`	`(pred: P, /, *R) -> O`	16
`LossOp[P, T = Container]`	`(pred: P, target: T, /) -> Tensor`	9
`MetricOp[P, T = Container]`	`(pred: P, target: T, /) -> Tensor`	2
`ExtendOp[P, O = P, **R = []]`	`(p: P, /, *R) -> O`	12
`ChooseOp`	`(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]`	6
`SearchOp[O, **R = []]`	`(index: Index, /, *R) -> O`	3
`WalkOp[T = PartialTree[Any, Any], O = Path, **R = []]`	`(x: T, /, *R) -> O`	3
`RewriteOp[M, O = M, **R = []]`	`(model: M, /, *R) -> O`	8
`TrainSetting[A, B, **R = []]`	`(x: A, /, *R) -> B`	6
STATEMENTS · commands		
`TrainCmd[X, **R = []]`	`(x: X, /, *R) -> None`	7
`OpCmd[C, A, **R = []]`	`(data: Dataset[C], /, *R) -> A`	12
`CodecCmd[O, **R = []]`	`(*R) -> O`	6
`SaveLoadCmd[O, **R = []]`	`(*R) -> O`	2
`CurateCmd[C, **R = []]`	`(data: Indexed[C], /, *R) -> Subset[C]` `indices(data: Indexed[C], /, *R) -> Tensor`	7
`PartitionCmd[C, **R = []]`	`(data: Indexed[C], /, *R) -> tuple[tuple[Subset[C], ...], ...]` `parts(data: Indexed[C], /, *R) -> tuple[tuple[Tensor, ...], ...]`	3

`NetworkCmd[O, **R = []]`	`(*R) -> O`	12
`DeployCmd[X, **R = []]`	`(x: X, /, *R) -> None`	7
`IndexCmd[O, **R = []]`	`(*R) -> O`	5

MISC

`MiscOp`	`(*args: Any, **named: Any) -> Any`	8
`MiscCmd`	`(*args: Any, **named: Any) -> Any`	1

OPS · values in, values out (a batch, a tree, a model as a value): nothing changes

`ConvertOp[A, B, **R = []]` A → B

`(x: A, /, *R) -> B`

A → B, anything in, anything out: what no narrower op says, such as mixing the examples of a batch or changing the container. ElementwiseOp (one example at a time) and LayerOp (a stateful module) are narrower kinds.

`Concat(ConvertOp[Seq, Seq])` Multimodal sequence — agnostic

`(*parts: Seq) -> Seq`

Several sequences of the same width as one (image patches, then caption tokens): one network reads them all.

`Transitions(ConvertOp[Path, Value])` Experience (transitions) — agnostic

`(path: Path, returns: Seq | None = None, /) -> Value`

The steps of walked paths as a set of transitions (state, action, reward, next state, done, and the action's log-probability), their order dropped: what a replay buffer holds and RL losses read.

`PathTo(ConvertOp[PartialTree[Any, Any], Path, [Tensor]])` A path from a tree new — agnostic

`(tree: PartialTree[Any, Any], leaf: Tensor, /) -> Path`

The steps from the root to a leaf of a tree's known part: the result of a walk.

`Mixup(ConvertOp[tuple[Container, Value], tuple[Container, Value]])` Data augmentation: mixup — agnostic

`(pair: tuple[Container, Value], /) -> tuple[Container, Value]`

SETTINGS `alpha: float = 0.2`

Each example blended with a random partner of the batch, labels by the same λ: the batch as a Set, paired at random.

`Cutmix(ConvertOp[tuple[Grid, Value], tuple[Grid, Value]])` Data augmentation: cutmix — agnostic

`(pair: tuple[Grid, Value], /) -> tuple[Grid, Value]`

A random box pasted from a partner image; the labels mix by its area.

`MeanBaseline(ConvertOp[Value, Value])` Baseline: the batch mean — agnostic

`(returns: Value, /) -> Value`

Each return minus the batch's mean: an advantage with less variance and no bias.

`GroupAdvantage(ConvertOp[Value, Value])` GRPO (group-relative advantages) — agnostic

`(rewards: Value, /) -> Value`

SETTINGS `n: int`

Each reward standardized within its group of n samples from one prompt: an advantage without a critic.

`QualityFilter(ConvertOp[Container, Container, [Value]])` Data filtering — agnostic

`[C: Container](x: C, scores: Value, /) -> C`

SETTINGS `threshold: float`

Keep the examples of a batch scoring above a threshold (a classifier's, a verifier's, a perplexity). Keeping the best fraction needs every batch: KeepBest.

`ElementwiseOp[A, B, **R = []]` example → example

`(x: A, /, *R) -> B`

A ConvertOp on one example at a time: output example i is made from input example i alone, reading no other example of the batch (an augmentation, a shift, a mask, filling holes, a scan along one trajectory).

`Patchify(ElementwiseOp[Grid, Grid])` Patching — agnostic

```
(x: Grid, /) -> Grid
SETTINGS size: int
```

An image → a coarser grid of p×p patches, each one element (a reshape); a linear map after it is a patch embedding.

`Flatten(ElementwiseOp[Grid, Seq])` Flattening — agnostic

```
(x: Grid, /) -> Seq
```

A grid → a sequence in raster order: the lattice is forgotten, learned positions say where each element was.

`RandomFlip(ElementwiseOp[Grid, Grid])` Data augmentation: random flip — agnostic

```
(x: Grid, /) -> Grid
SETTINGS p: float = 0.5
```

Mirror each image with probability p: its elements move, none changes.

`RandomCrop(ElementwiseOp[Grid, Grid])` Data augmentation: random crop — agnostic

```
(x: Grid, /) -> Grid
SETTINGS padding: int = 2
```

Pad, and cut back at a random offset: a random translation.

`FewShot(ElementwiseOp[Seq, Seq])` In-context learning (few-shot prompts) — agnostic

```
→ Concat
```

Solved examples, then the query, as one prompt: the model adapts by reading them.

`RoPE(ElementwiseOp[Tensor, Tensor, [Tensor]])` Positional encoding: rotary (RoPE) — agnostic

```
(x: Tensor, positions: Tensor, /) -> Tensor
```

Rotate pairs of query and key features by angles proportional to the position, so that attention depends only on relative offsets.

`VectorQuantize(ElementwiseOp[Container, Container, [Tensor]])` Vector quantization — agnostic

```
(x: Container, codebook: Tensor, /) -> Container
```

Each element → the id of its nearest codebook entry (real → classes), with a straight-through gradient.

`StraightThrough(ElementwiseOp[Tensor, Tensor, [Tensor]])` Straight-through estimator — agnostic

```
(hard: Tensor, soft: Tensor, /) -> Tensor
```

Forward the hard value, backpropagate through the soft one: a gradient for a choice that has none.

`Condition(ElementwiseOp[Container, Container, [Value]])` Conditioning — agnostic

```
(x: Container, label: Value, /) -> Container
```

Attach a label to every element (its one-hot vector next to each element's values; a none label for guidance).

`ColorJitter(ElementwiseOp[Grid, Grid])` Data augmentation: color jitter — agnostic

```
(x: Grid, /) -> Grid
SETTINGS strength: float = 0.2
```

Scale each image's brightness and contrast by random factors.

`GaussianNoise(ElementwiseOp[Container, Container])` Data augmentation: additive noise — agnostic

```
(x: Container, /) -> Container
SETTINGS sigma: float = 0.05
```

x + σ·ε at every element.

`LabelSmoothing(ElementwiseOp[Value, Value])` Label smoothing — agnostic

```
(y: Value, /) -> Value
SETTINGS eps: float = 0.1
```

A class → a distribution over classes, (1 − ε)·one_hot + ε / n: never pushed to infinite confidence.

`Attribution(ElementwiseOp[Container, Container, [Predictor]])` Attribution (input × gradient) — agnostic

```
(x: Container, model: Predictor, /) -> Container
```

How much each input element pushed the output: the input times the output's gradient, on any differentiable model.

`TokenDropout(ElementwiseOp[Seq, Seq])` Data augmentation: token dropout — agnostic

```
(x: Seq, /) -> Seq
SETTINGS p: float = 0.1
```

Drop a random fraction of the positions; the rest keep their order.

`ReturnsToGo(ElementwiseOp[Seq, Seq])` Returns-to-go agnostic

```
init() -> Tensor
step(state: Tensor, reward: Tensor, /) -> tuple[Tensor, Tensor]
SETTINGS gamma: float = 1.0
STATE the return from here on
```

G_t = r_t + γ·G_{t+1}: what each step led to, scanned from the end.

`Negamax(ElementwiseOp[Seq, Seq])` Self-play agnostic

```
init() -> Tensor
step(state: Tensor, reward: Tensor, /) -> tuple[Tensor, Tensor]
STATE the outcome for the player to move
```

One policy plays both sides; each move's return is the outcome for whoever made it: G_t = r_t − G_{t+1}.

`GAE(ElementwiseOp[Seq, Seq])` Value functions, Bellman, GAE agnostic

```
init() -> Tensor
step(state: Tensor, reward_value: tuple[Tensor, Tensor], /) -> tuple[Tensor, Tensor]
SETTINGS gamma: float = 0.99, lmbda: float = 0.95
STATE the advantage from here on
```

A critic's TD errors r + γV(s′) − V(s), averaged backward with weights (γλ)^k; each element a (reward, value).

`ParallelScan(ElementwiseOp[Seq, Seq])` Parallel scan agnostic

```
→ torch.cumsum
```

s_t = a_t·s_{t−1} + b_t for every t at once, in logarithmic depth: how a recurrence trains in parallel.

`NextToken(ElementwiseOp[Temporal, Example])` Self-supervised: next-token prediction agnostic

```
(x: Temporal, /) -> Example
```

Each element is predicted from the ones before it (a causal Temporal): the target at i is the element at i + 1.

`Mask(ElementwiseOp[Container, Example])` Self-supervised: masked prediction (BERT, MAE, MaskGIT) agnostic

```
(x: Container, /) -> Example
SETTINGS ratio: float | None = None
```

Random elements at level 1 ([MASK] for classes, 0 for real values): 15% is BERT, 75% MAE, a random ratio per example MaskGIT and masked diffusion.

`Noise(ElementwiseOp[Container, Example])` Diffusion / score matching / flow matching agnostic

```
(x: Container, /) -> Example
```

Every element at one random level t (x_t = (1 − t)·x + t·ε): the target is the velocity, the noise or the clean data, rescalings of one another.

`DiffusionForcing(ElementwiseOp[Temporal, Example])` Diffusion forcing new agnostic

```
(x: Temporal, /) -> Example
```

A random level for each element: next-token prediction and diffusion are its two extremes.

`NextFrame(ElementwiseOp[Temporal[Grid], Example])` Self-supervised: future-frame prediction agnostic

```
(x: Temporal[Grid], /) -> Example
```

The frames up to t as the input, the frames from t + 1 as the target: shift, on the frames of a video.

`Reconstruction(ElementwiseOp[Container, Example])` Reconstruction (autoencoder) agnostic

```
(x: Container, /) -> Example
```

Nothing hidden; the target is the input itself, so the model's bottleneck is what makes it hard.

`SFT(ElementwiseOp[Temporal, Example])` Instruction tuning / SFT agnostic

```
(x: Temporal, /) -> Example
SETTINGS prompt_len: int
```

shift, with only the response as targets: the prompt is read, never predicted.

`Fill(ElementwiseOp[Holes[Any], Container, [Predictor, Schedule]])` Fill new agnostic

```
[C: Container](holes: Holes[C], model: Predictor, order: Schedule, /) -> C
SETTINGS steps: int
STATE Holes[C] — the levels fall, as the order decides
```

Repeatedly predict the holes and lower their levels as the order decides (generation, inpainting, denoising), until every level is 0. With levels in (0, 1), what the model predicts is the flow back to the data. steps is how many predictions. Holes are a sequence of masks and one of elements: a sequence in, a sequence out.

`LayerOp[A, B]` A → B (an nn.Module)

```
(x: A, /) -> B
```

A layer: an expression that is a stateful nn.Module, (input) → output, its parameters its state. Making one allocates them (sizes it is not given are read from its first input, as torch's lazy modules do); calling it reads them; commands change them (TrainCmd, DeployCmd). Layer wraps any torch module; the methods below are the layers torch does not have.

`Layer(LayerOp[Any, Any])` A torch layer (any nn.Module) model

```
(x: Any, /) -> Any
SETTINGS module: nn.Module
```

Any torch module as a layer: Layer(nn.Linear(64, 10)), Layer(nn.Conv2d(3, 64, 3)), Layer(nn.GRU(64, 64)). A linear model is Layer(nn.LazyLinear(n)): linear or logistic regression by the loss its target picks, an SVM with Hinge, closed form with LeastSquares.

`Network(LayerOp[Container, Prediction])` Network (each layer chosen by the types) model

```
(x: Container, /) -> Prediction
SETTINGS target: Any, d: int = 64, depth: int = 2, mixer: LayerOp[Any, Any] | None = None, mlp:
LayerOp[Any, Any] | None = None
```

embed → position → [aggregate, MLP] × depth → pool → readout, each slot chosen from the input type (at the first call) and the target's (Embed, AutoAggregate, Pool, Readout); mixer and mlp replace a slot's choice with a layer (a scan, experts).

`Encoder(LayerOp[Container, Value])` Embedding models (a container to one vector) model

```
(x: Container, /) -> Value
SETTINGS dim: int
```

A network from any container to one vector per example: the towers of CLIP, a reward model, a critic.

`Embed(LayerOp[Container, Container])` Embeddings model

```
(x: Container, /) -> Container
SETTINGS d: int
```

By what the elements hold: classes → a row of a lookup table (one more for [MASK]); real values → a linear map.

`Readout(LayerOp[Container, Prediction])` Readout model

```
(h: Container, /) -> Prediction
SETTINGS target: Any, d: int
```

By what the target holds: a logit per class, a value per real number, one readout per field of a record.

`LearnedPositions(LayerOp[Container, Container])` Positional encoding: learned model

```
(x: Container, /) -> Container
SETTINGS d: int
```

A learned vector added at each position of every Seq level (Temporal, Spatial: a Grid has two); a Set, a Graph or a Tree gets none.

`SinusoidalPositions(LayerOp[Container, Container])` Positional encoding: sinusoidal model

```
(x: Container, /) -> Container
SETTINGS d: int
```

Fixed sin/cos features of each position at geometrically spaced frequencies, added.

`MoE(LayerOp[Container, Container])` Mixture of experts model

```
(x: Container, /) -> Container
SETTINGS d: int, experts: int = 8, k: int = 2
```

A router sends each element to its top-k experts (MLPs) and sums their gate-weighted outputs.

`Autoencoder(LayerOp[Container, Container])` Latent space / bottleneck model

```
(x: Container, /) -> Container
encode(x: Container, /) -> Container
decode(z: Container, /) -> Container
SETTINGS latent: int
```

encode to a smaller container (fewer positions, few channels), decode back: the latent must keep what matters, and generation can happen there (latent diffusion). Its .encode and .decode are the two halves.

`SAE(LayerOp[Container, Container])` Sparse autoencoders (superposition) model

```
(x: Container, /) -> Container
SETTINGS d: int, features: int
```

Activations → a wide, sparse code → activations, trained by reconstruction plus an L1 penalty: each code unit tends to be one feature.

RandomFourierFeatures(LayerOp[Value, Value]) Kernel methods (random features) new model

```
(x: Value, /) -> Value
SETTINGS features: int = 1024, bandwidth: float = 1.0
```

Random Fourier features of a shift-invariant kernel (RBF by default): a linear layer on them is a kernel machine, so kernel ridge regression is LeastSquares on it and a kernel SVM is Hinge on it.

Residual(LayerOp[Container, Container]) Residual connections model

```
(h: Container, /) -> Container
SETTINGS f: nn.Module
```

h + f(norm h): the layer edits the stream instead of replacing it.

AutoAggregate(LayerOp[Container, Container]) Choosing the aggregation from the container model

```
(x: Container, /) -> Container
SETTINGS d: int
```

What each element reads, level by level of the container: Graph, Tree → message passing · causal Temporal → causal attention (a selective scan when long) · Spatial (a large Grid) → convolution · several levels → axial · else attention over all.

SSM(LayerOp[Seq, Seq]) State-space models / linear recurrences new model

```
(x: Seq, /) -> Seq
SETTINGS d: int
STATE s_t = a_t·s_{t−1} + b_t, run as a parallel scan
```

A linear recurrence whose coefficients come from the input (S4, Mamba, linear attention): every earlier element read, in linear time.

GNN(LayerOp[Graph, Graph]) Message passing (GNN) model

```
(x: Graph, /) -> Graph
SETTINGS d: int
```

Each node reads its own features and the mean along its incoming edges; in a tree, children pass up to parents.

Axial(LayerOp[Container, Container]) Axial (nested structures) model

```
(x: Container, /) -> Container
SETTINGS d: int
```

One aggregation per level of a nested container (a video's frames, then its rows, then its columns), the other levels as batch; summed.

UNet(LayerOp[Grid, Grid]) Encoder–decoder hierarchy (U-Net) model

```
(x: Grid, /) -> Grid
SETTINGS d: int, levels: int = 2
```

Down to a coarse grid and back up, with skip connections: each element reads a wide neighbourhood.

RetrievalAttention(LayerOp[tuple[Container, Index], Container]) Retrieval (attention over a memory) model

```
(x: tuple[Container, Index], /) -> Container
SETTINGS k: int
```

Each element reads the k nearest entries of an external memory (an Index), softmax-weighted.

Pool(LayerOp[Container, Value]) Pooling (mean / last position) model

```
(x: Container, /) -> Value
```

A container's features → one vector per example: a causal Temporal keeps its last position, the one that read everything; anything else, the mean.

ReduceOp[C, A, **R = []] Set / Seq → element

```
(x: C, /, *R) -> A
```

Reduce a set or a sequence C to one element A, in order (left to right, as functools.reduce).

DiscountedReturn(ReduceOp[Path, Value]) Expected return (the RL objective) agnostic

```
(path: Path, /) -> Value
SETTINGS gamma: float = 1.0
```

Σ γ^t·r_t along each path: what RL maximizes. A walk can't be differentiated through, so its gradient comes from scoring the path (reinforce).

BestOfN(ReduceOp[Path, Path, [Value]]) Best-of-N with a verifier agnostic

```
(paths: Path, scores: Value, /) -> Path
SETTINGS n: int
```

The best-scoring of each group of n paths (scored by a verifier or a reward model).

MajorityVote(ReduceOp[Value, Value]) Self-consistency (majority vote) agnostic

```
(answers: Value, /) -> Value
SETTINGS n: int
```

The most frequent answer of each group of n samples.

ScalingLaw(ReduceOp[Seq, tuple[float, float], [Seq]]) Scaling laws (power-law fit) agnostic

```
(sizes: Seq, losses: Seq, /) -> tuple[float, float]
```

loss ≈ a·size^(−b), fitted on the logs.

ScoreOp[P, O = Tensor, **R = []] element → scalar

```
(pred: P, /, *R) -> O
```

How bad a prediction P is (against R: a target, a weight): every loss, every surrogate objective, every metric. O is a number to minimize (a Tensor), or one per example (a Value).

Lagrangian(ScoreOp[Seq, Seq]) Constrained objectives (Lagrangian) agnostic

```
init() -> float
step(state: float, losses: tuple[Tensor, Tensor], /) -> tuple[float, Tensor]
SETTINGS lr: float = 0.1
STATE the multiplier λ ≥ 0
```

(loss f, constraint g) in, f + λ·(g − c) out; λ rises while the constraint is violated (dual ascent).

Objective(ScoreOp[nn.Module, Tensor, ...]) Objective (a split, then its loss) new agnostic

```
(model: nn.Module, /, *batch: Any) -> Tensor
SETTINGS split: Callable[..., Example] | None = None
```

The loss of a model on a batch: split it into an Example with a data op (NextToken, Mask, Noise, …), predict from the input, score with the loss the target picks. With no split, the batch is (input, target) already.

LogProb(ScoreOp[Prediction, Value, [Container]]) Log-probability new agnostic

```
(pred: Prediction, x: Container, /) -> Value
```

log p(x) for each element under a prediction, unreduced: what losses and estimators are built from.

PathLogProb(ScoreOp[PartialTree[Any, Any], Value, [Path]]) Teacher forcing (the log-probability of a path)

```
(tree: PartialTree[Any, Any], path: Path, /) -> Value
```

agnostic

Σ log options(node)(edge) along a path; on the data's own path, −1/length of it is the next-token loss.

SequenceLogProb(ScoreOp[Predictor, Value, [Temporal]]) Sequence log-probability agnostic

```
(model: Predictor, x: Temporal, /) -> Value
SETTINGS start: int = 1
```

log p(x[start:] | the rest before it), per sequence: teacher forcing on whole sequences, in one pass.

DPO(ScoreOp[nn.Module, Tensor, ...]) Preferences (DPO) agnostic

```
(model: nn.Module, /, *batch: Any) -> Tensor
SETTINGS reference: Predictor, beta: float = 0.1
```

The loss of (policy, chosen, rejected): Bradley–Terry on β·(the policy's log-ratio to the reference, chosen minus rejected). Preferences as classification: no reward model, no walk.

KLPenalty(ScoreOp[Value, Value, [Value, Value]]) KL-regularized reward (RLHF) agnostic

```
(reward: Value, LogProb: Value, ref_log_prob: Value, /) -> Value
SETTINGS beta: float = 0.1
```

The reward, minus how far the policy has moved from a reference: use it as the return.

ELBO(ScoreOp[Model[Container, Dist], Tensor, [Model[Tensor, Prediction], Container]]) VAE (evidence lower bound) agnostic

```
(encoder: Model[Container, Dist], decoder: Model[Tensor, Prediction], x: Container, /) -> Tensor
```

The reconstruction from a latent drawn from the posterior (rsample: reparameterization), plus KL(posterior ‖ prior).

GAN(ScoreOp[Model[Tensor, Container], tuple[Tensor, Tensor], [Predictor, Container, Tensor]]) GAN (two players) agnostic

```
(generator: Model[Tensor, Container], discriminator: Predictor, real: Container, z: Tensor, /) ->
tuple[Tensor, Tensor]
```

The discriminator tells real from generated; the generator wants its samples called real. Two scores, for two players (alternate their steps).

FlowLogProb(ScoreOp[Container, Value, [list[Any]]]) Normalizing flows model

```
(x: Container, transforms: list[Any], /) -> Value
```

The exact log-likelihood through invertible maps (change of variables).

`Reinforce(ScoreOp[Path, Tensor, [Value]])` Policy gradients (REINFORCE) agnostic

```
(path: Path, advantage: Value, /) -> Tensor
```

−E[log p(path)·A]: a walked path's log-probability, weighted by how much better than usual it went.

`PPO(ScoreOp[Value, Tensor, [Value, Value]])` PPO clipped surrogate agnostic

```
(LogProb: Value, old_log_prob: Value, advantage: Value, /) -> Tensor
SETTINGS clip: float = 0.2
```

The policy gradient with the ratio π / π_old clipped: it stops pushing once the policy has moved far.

`Pathwise(ScoreOp[Dist, Tensor, [Model[Tensor, Tensor]]])` Pathwise estimator (through a critic) agnostic

```
(policy: Dist, critic: Model[Tensor, Tensor], /) -> Tensor
```

−E[Q(a)] with a = rsample(): differentiated through the sample into a learned critic (DDPG, SAC).

`EntropyBonus(ScoreOp[Dist])` Maximum-entropy RL (entropy bonus) agnostic

```
(options: Dist, /) -> Tensor
SETTINGS alpha: float = 0.01
```

−α·H(options): keeps the policy exploring.

`MAML(ScoreOp[nn.Module, Tensor, [Callable[..., Tensor], Container, Container]])` Meta-learning (MAML)

```
(model: nn.Module, objective: Callable[..., Tensor], support: Container, query: Container, /) -> Tensor
```

agnostic

Bilevel: the query loss of the weights after an inner step (an unroll) on the support set, differentiated through.

`Grad(ScoreOp[Tensor, Tensor])` Backpropagation / autodiff agnostic

```
→ torch.autograd
```

The gradient of any score.

`LossOp[P, T = Container]` (input, target) → ℝ

```
(pred: P, target: T, /) -> Tensor
```

A loss: how far a prediction is from its target, (input, target) → a number to minimize, differentiable.

`Loss(LossOp[Prediction])` Loss = negative log-likelihood of the target agnostic

```
(pred: Prediction, target: Container, weight: Tensor | None = None, /) -> Tensor
```

−log p(target), chosen by what the target holds: classes → cross-entropy · bits → BCE · real → squared error · distribution → KL · record → the sum of its fields' losses.

`CrossEntropy(LossOp[Tensor])` Cross-entropy agnostic

```
(logits: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor
```

Classes, or a distribution over them (soft labels).

`BinaryCrossEntropy(LossOp[Tensor])` Binary cross-entropy agnostic

```
(logits: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor
```

Bits.

`Hinge(LossOp[Tensor, Value])` Hinge loss (SVM) new agnostic

```
(scores: Tensor, target: Value, weight: Tensor | None = None, /) -> Tensor
SETTINGS margin: float = 1.0
```

max(0, margin − y·f(x)) for labels ±1 (for classes: the margin over the best wrong class): a support vector machine is a linear layer trained with it (on RandomFourierFeatures for a kernel SVM).

`MSE(LossOp[Tensor])` Squared error agnostic

```
(pred: Tensor, target: Container, weight: Tensor | None = None, /) -> Tensor
```

Real values: a unit Gaussian's negative log-likelihood, up to a constant.

`KL(LossOp[Dist, Dist])` KL divergence agnostic

```
(pred: Dist, target: Dist, weight: Tensor | None = None, /) -> Tensor
```

A distribution against a reference (a prior, a reference policy).

`MultiTaskLoss(LossOp[Prediction])` Multi-task training agnostic

```
(pred: Prediction, target: Container, weight: Tensor | None = None, /) -> Tensor
```

A record: one loss per field, each chosen by what the field holds, summed.

`InfoNCE(LossOp[Value, Value])` Contrastive (InfoNCE / CLIP) agnostic

```
(a: Value, b: Value, /) -> Tensor
SETTINGS temperature: float = 0.07
```

Which b goes with each a, among the batch: cross-entropy over similarities, the matching pair as the class.

`BradleyTerry(LossOp[Value, Value])` Reward models (Bradley–Terry) agnostic

```
(chosen: Value, rejected: Value, /) -> Tensor
```

−log σ(r(chosen) − r(rejected)): the likelihood that the preferred answer wins.

`MetricOp[P, T = Container]` (input, target) → ℝ

```
(pred: P, target: T, /) -> Tensor
```

A metric: how good a prediction is against its target, (input, target) → a number, reported and selected on, never differentiated (a loss stands in for it in training).

`Accuracy(MetricOp[Tensor])` Metrics (accuracy) agnostic

```
(pred: Tensor, target: Container, /) -> Tensor
```

Reported and selected on, never differentiated: a loss stands in for it in training.

`RetrievalAccuracy(MetricOp[Value, Value])` Retrieval accuracy new agnostic

```
(a: Value, b: Value, /) -> Tensor
```

How often each a is closest to its own b among the batch (matching images to captions).

`ExtendOp[P, O = P, **R = []]` Tree → Tree

```
(p: P, /, *R) -> O
```

How a partial container P goes on: its options, its model, what its nodes keep. A P in, a P out (an unroll: the list it stands for).

`Unroll(ExtendOp[PartialSeq[Any], list[Any]])` Unroll agnostic

```
(seq: PartialSeq[Any], /) -> list[Any]
SETTINGS steps: int
STATE S — any state
```

The list a PartialSeq stands for: start, step(start), step(step(start)), … steps of them.

`Iteration(Unroll)` Iteration (rounds of a program) agnostic

```
→ repeat
SETTINGS steps: int
STATE S — any state
```

Rounds of collecting and learning (policy iteration, self-play, expert iteration): a script's repeat (program.py), since each round's fit is a statement.

`Temperature(ExtendOp[PartialTree[Any, Any]])` Sampling temperature agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS t: float
```

The options' logits divided by t: below 1 sharper (toward greedy), above 1 flatter.

`TopK(ExtendOp[PartialTree[Any, Any]])` Top-k sampling agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS k: int
```

Only the k likeliest edges stay possible.

`TopP(ExtendOp[PartialTree[Any, Any]])` Top-p (nucleus) sampling agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS p: float
```

Only the likeliest edges whose probabilities add up to p stay possible.

`EpsilonGreedy(ExtendOp[PartialTree[Any, Any]])` Exploration vs exploitation (ε-greedy) agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS eps: float
```

With probability ε a uniform edge, else the likeliest.

`ToolUse(ExtendOp[PartialTree[Any, Any], PartialTree[Any, Any], [Callable[[Any], Tensor]]])` Tool use agnostic

```
(tree: PartialTree[Any, Any], tool: Callable[[Any], Tensor], /) -> PartialTree[Any, Any]
SETTINGS call: int
```

When a walk emits the call token, follow writes the tool's answer (a calculator, a search, an interpreter).

`KVCache(ExtendOp[PartialTree[Any, Any]])` KV caching new agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
STATE each known node keeps its keys and values
```

The tree's known nodes keep what the model computed for them, so expanding a node costs one position.

`WorldModel(ExtendOp[PartialTree[Any, Any], PartialTree[Any, Any], [Model[Container, Container]]])` World models (a learned follow) agnostic

```
(tree: PartialTree[Any, Any], world: Model[Container, Container], /) -> PartialTree[Any, Any]
```

The environment's follow (and reward, done) replaced by a learned model of them: walks, searches and plans in imagination (Dreamer, MuZero).

`MPC(ExtendOp[PartialTree[Any, Any]])` Planning (random shooting / MPC) agnostic

```
(tree: PartialTree[Any, Any], world: PartialTree[Any, Any] | None = None, /) -> PartialTree[Any, Any]
SETTINGS horizon: int = 5, samples: int = 64
STATE the random plans tried from each node
```

Options that look ahead: random action sequences simulated in a world, all the probability on the first action of the best.

`MCTS(ExtendOp[PartialTree[Any, Any]])` MCTS (tree search) agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS simulations: int = 64, two_player: bool = False
STATE a search tree per node: priors, visit counts, values
```

Options improved by search: grow a tree by PUCT (the options as priors), back up rewards and values, return the visit counts. Walk it to search; train toward it to amortize the search (AlphaZero).

`KNNLM(ExtendOp[PartialTree[Any, Any], PartialTree[Any, Any], [Index, Predictor]])` kNN-LM (a memory of contexts) new agnostic

```
(tree: PartialTree[Any, Any], memory: Index, embed: Predictor, /) -> PartialTree[Any, Any]
SETTINGS weight: float = 0.25
```

Options mixed with the next tokens of the k nearest stored contexts: an LM that can look things up.

`ChooseOp` Tree → next

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
```

A walk's step: from the tree expanded so far, the next (which edges to follow, which leaves to keep).

`Sample(ChooseOp)` Sampling new agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
STATE PartialTree — one leaf per root
```

Expand each kept leaf by one edge drawn from its options.

`Greedy(ChooseOp)` Greedy decoding agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
STATE PartialTree — one leaf per root
```

Expand each kept leaf by its likeliest edge.

`Beam(ChooseOp)` Beam search agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS k: int
STATE k leaves per root and their log-probabilities
```

Expand every kept leaf; keep the k likeliest children of each root.

`NPaths(ChooseOp)` Several paths per root (best-of-N, groups) new agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS n: int, choose: Chooser
STATE n independent leaves per root
```

n paths from each root side by side, each grown by its own chooser: n samples per prompt (best-of-N, GRPO).

`TeacherForcing(ChooseOp)` Teacher forcing (walking the data's path) new agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS edges: Seq
STATE one leaf per root, on the data's path
```

Expand each leaf by the next given edge: scoring the path walked is teacher forcing.

`Speculative(ChooseOp)` Speculative decoding agnostic

```
(tree: PartialTree[Any, Any], /) -> PartialTree[Any, Any]
SETTINGS draft: PartialTree[Any, Any], k: int = 4
STATE the accepted leaf, and a draft model's guesses past it
```

The draft walks k edges ahead greedily, the tree's model checks them in one pass, the agreeing prefix is kept: greedy decoding, in fewer of the big model's passes.

`SearchOp[O, **R = []]` Index × element → Set

```
(index: Index, /, *R) -> O
```

What is built on looking up an index (index.search): the k elements nearest each query, put to use as an O.

`Retrieve(SearchOp[Temporal, [Set, Temporal, Predictor]])` Retrieval-augmented generation (retrieve) new

agnostic

```
(index: Index, documents: Set, query: Temporal, embed: Predictor, /) -> Temporal
SETTINGS k: int = 4
```

The k documents nearest the query's embedding, put in front of it as one prompt (then walk the LM's tree).

`HardNegatives(SearchOp[Set, [Value, Value]])` Hard-negative mining new

agnostic

```
(index: Index, queries: Value, positives: Value, /) -> Set
SETTINGS k: int = 8
```

The nearest elements that are not the match: harder negatives for a contrastive loss.

`KNN(SearchOp[Prediction, [Value, Value]])` k-nearest neighbours (classification, regression) new

agnostic

```
(index: Index, queries: Value, labels: Value, /) -> Prediction
SETTINGS k: int = 5
```

k-nearest-neighbour prediction: each query's k nearest stored examples (index.search), their labels voted (classes) or averaged (real values), weighted by distance. Nothing to fit: Insert the examples, then ask.

`WalkOp[T = PartialTree[Any, Any], O = Path, **R = []]` Tree → Path

```
(x: T, /, *R) -> O
```

Grow a tree and read the paths it walked: one path, a batch of them (beam search, n per root), or walks of a live policy in an environment.

`Walk(WalkOp[PartialTree[Any, Any], Path, [Chooser]])` Walk new

agnostic

```
(tree: PartialTree[Any, Any], choose: Chooser, /) -> Path
SETTINGS steps: int
STATE PartialTree — its known part, grown by the chooser
```

Grow the known part of a tree from its roots, steps times, a chooser deciding which edges to follow and which leaves to keep; then the path to a kept leaf. Sampling, greedy decoding, beam search, best-of-N and RL rollouts are walks with different choosers.

`ChainOfThought(Walk)` Chain-of-thought / longer reasoning

agnostic

```
→ Walk
SETTINGS steps: int
STATE PartialTree — its known part, grown by the chooser
```

A longer walk before the answer; the gains come from training the walk (R1).

`Collector(WalkOp[Environment, Dataset[Path], [Predictor]])` Collector (walks of a live policy) new

agnostic

```
(env: Environment, policy: Predictor, /) -> Dataset[Path]
SETTINGS steps: int, batch: int
```

Walks of the environment with the policy as it is when each batch is drawn: on-policy data, read inside the fit that changes the policy (torchrl's collectors).

`RewriteOp[M, O = M, **R = []]` Model → Model′

```
(model: M, /, *R) -> O
```

A model M seen another way: a view that reads its parameters and adds none (observe, patch, steer, compile), or a model built from it (a condition fixed, guidance). M stays as it was.

`Given(RewriteOp[Predictor, Predictor, [Value | None]])` A condition, fixed new

agnostic

```
(model: Predictor, condition: Value | None, /) -> Predictor
```

A conditional model with its condition fixed (None: no condition): the two models guidance mixes.

`CFG(RewriteOp[Predictor, Predictor, [Predictor]])` Classifier-free guidance

agnostic

```
(cond: Predictor, uncond: Predictor, /) -> Predictor
SETTINGS w: float
```

u + w·(c − u): extrapolate from the unconditional prediction toward the conditional one, for a tree's options or a fill's model.

`CFGDropout(RewriteOp[Predictor])` Classifier-free guidance training new

agnostic

```
(model: Predictor, /) -> Predictor
SETTINGS p: float = 0.1
```

The condition dropped with probability p while training, so one model is both conditional and unconditional.

`Observe(RewriteOp[nn.Module, nn.Module, [nn.Module]])` Reading activations

agnostic

```
(model: nn.Module, at: nn.Module, /) -> nn.Module
```

A model that returns what one of its modules computed: the activations probes, SAEs and circuits study.

Patch(RewriteOp[nn.Module, nn.Module, [nn.Module, Callable[[Tensor], Tensor]]]) Activation patching (circuits) agnostic

```
(model: nn.Module, at: nn.Module, replace: Callable[[Tensor], Tensor], /) -> nn.Module
```

A model with one module's output replaced (by another input's activations): which components carry a behaviour?

Steer(RewriteOp[nn.Module, nn.Module, [nn.Module, Tensor]]) Steering along a direction agnostic

```
(model: nn.Module, at: nn.Module, direction: Tensor, /) -> nn.Module
SETTINGS alpha: float = 1.0
```

α·direction added to one module's output: features are directions, and moving along one changes the behaviour.

Autocast(RewriteOp[nn.Module, nn.Module]) Low precision: training agnostic

```
→ torch.autocast
```

bf16 (or fp8) compute, float32 master weights.

Compile(RewriteOp[nn.Module, nn.Module]) Hardware-aware kernels model

```
→ torch.compile
```

torch.compile(module) fuses its ops; attention picks flash kernels by itself.

TrainSetting[A, B, **R = []] a setting of TrainCmd

```
(x: A, /, *R) -> B
```

A part of training that TrainCmd carries from step to step: an optimizer, a learning-rate schedule, parameter groups, a gradient-free update. A value, given to TrainCmd as a setting.

DecayGroups(TrainSetting[nn.Module, list[dict[str, Any]]]) Weight decay (parameter groups) agnostic

```
(model: nn.Module, /) -> list[dict[str, Any]]
SETTINGS weight_decay: float = 0.1
```

The parameters to decay (the weight matrices) apart from the rest (norms, biases, embeddings).

MuP(TrainSetting[nn.Module, list[dict[str, Any]]]) Parameterization (μP) agnostic

```
(model: nn.Module, /) -> list[dict[str, Any]]
SETTINGS width: int, base_width: int
```

Hidden matrices reweighted by base_width / width, so a learning rate tuned narrow carries over wide.

Optimizer(TrainSetting[Seq, Seq]) Optimizers (SGD, momentum, Adam, Muon) new agnostic

```
init() -> Tensor
step(state: Tensor, grad: Tensor, /) -> tuple[Tensor, Tensor]
SETTINGS kind: str = "adamw", lr: float = 0.0003
STATE the moments of every parameter
```

Gradients in, updates out: SGD carries nothing, momentum a velocity, Adam two moments, Muon orthogonalizes. A setting of Train, which carries its state from step to step.

WarmupCosine(TrainSetting[Seq, Seq]) Learning-rate schedules agnostic

```
init() -> int
step(state: int, t: int, /) -> tuple[int, float]
SETTINGS warmup: int, total: int
STATE the step count
```

A learning rate for each step: linear warmup, then cosine decay. A setting of Train.

Langevin(TrainSetting[Container, Container]) Energy-based models (Langevin) agnostic

```
(state: Container, /) -> Container
SETTINGS energy: Predictor, step_size: float = 0.01
STATE a sample, moving
```

$x \leftarrow x - \eta \cdot \nabla E(x) + \sqrt{(2\eta)} \cdot \varepsilon$: samples of exp(−energy), with the gradient from autograd.

EvolutionStrategy(TrainSetting[Dist, Dist]) Evolution strategies (no gradient) agnostic

```
(state: Dist, /) -> Dist
SETTINGS fitness: Callable[[Tensor], Tensor], population: int = 32
STATE a search distribution over the parameters
```

Draw a population (a Set of parameters) from the search distribution, score them, move it toward the better ones: it works when nothing is differentiable (CMA-ES keeps a covariance too).

COMMANDS · make new state, change it in place (!), or read a whole dataset

`TrainCmd[X, **R = []]` X! × …

```
(x: X, /, *R) -> None
```

Fit an X to data, in place: a model by a loss's gradient, a closed form or an average, an estimator by its own rule.

`Train(TrainCmd[nn.Module, [Callable[..., Tensor], Dataset[Any], Dataset[Any] | None]])` Training new

```
(model: nn.Module, objective: Callable[..., Tensor], data: Dataset[Any], valid: Dataset[Any] | None = None, /) -> None
SETTINGS steps: int, optimizer: Scan[Tensor, Any, Tensor] = Optimizer(kind='adamw', lr=0.0003), schedule: Scan[int, int, float] | None = None, max_grad_norm: float | None = 1.0, precision: torch.dtype | None = None, patience: int | None = None, every: int = 100
```

agnostic

Descend the objective over the dataset's batches: each step, objective(model, *batch) (under autocast to precision), its gradient (clipped to max_grad_norm), the optimizer's update of the parameters, in place, at the schedule's rate. In training mode only here, so a forward's own effects (BatchNorm statistics) are training's too. Given valid and patience, every `every` steps the objective's mean on valid is checked, and training ends early once it has not improved for `patience` checks. Reinforcement learning has a loop of its own: TrainRL.

`TrainRL(TrainCmd[nn.Module, [Environment, Callable[..., Tensor]]])` Training (reinforcement learning) new

```
(policy: nn.Module, env: Environment, objective: Callable[..., Tensor], /) -> None
SETTINGS steps: int, horizon: int = 200, walks: int = 16, buffer: int | None = None, optimizer: Scan[Tensor, Any, Tensor] = Optimizer(kind='adamw', lr=0.0003), max_grad_norm: float | None = 1.0
```

agnostic

Reinforcement learning's loop: each step, walk the environment with the policy as it is now (a collector), keep the walks for one update (on-policy) or in a replay buffer of at most buffer transitions (off-policy), then a gradient step on objective(policy, batch). Not Train: its data is made by the model being trained, and lives as long as the collector or the buffer keeps it, not as long as a fixed dataset.

`LeastSquares(TrainCmd[nn.Module, [Dataset[Any]]])` Least squares (closed form, ridge) new

```
(model: nn.Module, data: Dataset[Any], /) -> None
SETTINGS ridge: float = 0.0
```

model

Solve for a linear layer's weights in closed form, in place: $(X^TX + \lambda I)^{-1}X^Ty$, X^TX and X^Ty accumulated over the dataset's batches; ridge regression when ridge is above 0.

`FitEstimator(TrainCmd[SkModel[Any], [Dataset[Any]]])` Fitting an estimator (fit, partial_fit) new

```
(model: SkModel[Any], data: Dataset[Any], /) -> None
SETTINGS incremental: bool | None = None
```

agnostic

Fit an estimator's model to a dataset, in place: batch by batch by partial_fit when incremental (by default, whenever the estimator has it), otherwise every batch gathered and fit once. Each batch is (features, target), or features alone for an unsupervised one.

`TrainingStep(TrainCmd[nn.Module])` Training step (clipping, mixed precision)

```
→ Train
```

agnostic

Train's settings: autocast the objective, clip the gradient norm.

`TwoPlayerGames(TrainCmd[nn.Module])` Two-player games (alternating fits)

```
→ repeat, Train
```

agnostic

No single quantity goes down: each player's Train on its own loss, in turn, inside a repeat (GANs, adversarial training).

`EMA(TrainCmd[Weights, [Weights]])` EMA of weights

```
(average: Weights, model: Weights, /) -> None
SETTINGS decay: float = 0.999
```

agnostic

average ← decay·average + (1 − decay)·model, in place: after each round or step, a smoother copy, and the slow target of self-distillation and DQN.

`OpCmd[C, A, **R = []]` Dataset → a

```
(data: Dataset[C], /, *R) -> A
```

An op over a whole dataset of C: a metric over every batch, a statistic, a map over every batch.

MapBatches(OpCmd[Any, Dataset[Any], [Callable[..., Any]]]) An expression on each batch new agnostic
`(data: Dataset[Any], f: Callable[..., Any], /) -> Dataset[Any]`
f applied to every batch, once per batch as the dataset batches it (never re-batched, since f may depend on what a batch holds): how a statement reads an expression's output, embeddings to index, features to probe.

Evaluate(OpCmd[Any, Tensor, [Callable[..., Tensor]]]) Evaluation (a metric over a dataset) new agnostic
`(data: Dataset[Any], metric: Callable[..., Tensor], model: nn.Module | None = None, /) -> Tensor`
A per-batch metric (accuracy, a loss, perplexity), averaged over every batch, weighted by their sizes; given a model, the metric is called as metric(model, *batch), as a loss is.

Metric(OpCmd[tuple[Tensor, Tensor], Any]) Whole-dataset metric new agnostic
`(data: Dataset[tuple[Tensor, Tensor]], /) -> Any`
SETTINGS `fn: Callable[[Any, Any], Any]`
A metric of every prediction and target at once, not a mean of per-batch values: data is (prediction, target) pairs, gathered, then fn(targets, predictions), as scikit-learn's metrics take them.

ClassificationMetrics(Metric) Classification metrics (AUC, F1, precision / recall, confusion matrix) new
→ `sklearn.metrics.roc_auc_score, f1_score, precision_recall_fscore_support, confusion_matrix` agnostic
SETTINGS `fn: Callable[[Any, Any], Any]`
The ones that need every score ranked or counted together: ROC AUC, F1, precision and recall, a confusion matrix.

Streaming(OpCmd[tuple[Tensor, Tensor], Tensor]) Streaming metric new agnostic
`(data: Dataset[tuple[Tensor, Tensor]], /) -> Tensor`
SETTINGS `metric: StreamMetric`
The same number as Metric without keeping every prediction: the metric is reset, updated on each (prediction, target) batch, then computed (torchmetrics' API).

StreamingMetrics(Streaming) Calibration error, FID (streaming) new agnostic
→ `torchmetrics.classification.CalibrationError, torchmetrics.image.fid.FrechetInceptionDistance`
SETTINGS `metric: StreamMetric`
Metrics of a whole dataset that keep running totals instead of every prediction: calibration error, FID.

FitPredict(OpCmd[Any, Tensor]) Fit and predict (transductive) new agnostic
`(data: Dataset[Any], /) -> Tensor`
SETTINGS `estimator: Estimator`
A fresh clone of the estimator fitted to the whole dataset, and what it says of each example: for methods that cannot label new data, only the data they were fitted to (fit_predict).

DBSCAN(FitPredict) Density clustering (DBSCAN, HDBSCAN) new agnostic
→ `sklearn.cluster.DBSCAN, HDBSCAN, OPTICS`
SETTINGS `estimator: Estimator`
Clusters as dense regions, how many found rather than given, some examples left as noise.

HierarchicalClustering(FitPredict) Hierarchical and spectral clustering new agnostic
→ `sklearn.cluster.AgglomerativeClustering, SpectralClustering`
SETTINGS `estimator: Estimator`
Clusters merged bottom up, the closest pair first, or cut from a graph of similarities (spectral).

FitTransform(OpCmd[Any, Tensor]) Fit and transform (transductive) new agnostic
`(data: Dataset[Any], /) -> Tensor`
SETTINGS `estimator: Estimator`
A fresh clone of the estimator fitted to the whole dataset, and each example's new features: for embeddings that place only the data they were fitted to (fit_transform).

Embedding2D(FitTransform) Neighbour embeddings (t-SNE, UMAP) new agnostic
→ `sklearn.manifold.TSNE, umap.UMAP`
SETTINGS `estimator: Estimator`
A low-dimensional map that keeps each example's neighbours near it: for looking at data, not for features.

Accumulate(OpCmd[Any, Any, [Scan[Any, Any, Any]]]) Accumulate (a scan across batches) new agnostic
`(data: Dataset[Any], s: Scan[Any, Any, Any], /) -> Any`
A Scan's state carried across a dataset's batches, its final state returned: normalization statistics, a histogram, any metric that is not a mean.

CodecCmd[O, **R = []] ... → Codec | Dataset

`(*R) -> O`

Data in and out: a codec (raw data in a vocabulary, and back), fitting one, batches from memory.

`Tokenizer(CodecCmd[Codec[list[str], Temporal]])` Tokenization (BPE, SentencePiece, bytes) new agnostic

```
() -> Codec[list[str], Temporal]
SETTINGS kind: str = "bpe", vocab: int = 32000
```

Text ⇄ a sequence of ids, built empty: BPE merges, fitted to a corpus by FitCodec (bytes alone: a fixed vocabulary of 256, nothing to fit). The tokenizer decides the content type: Temporal of classes(vocab).

`ImageTokenizer(CodecCmd[Codec[Grid, Grid], [Grid]])` Image tokenizer (VQ-VAE, VQGAN) new agnostic

```
(images: Grid, /) -> Codec[Grid, Grid]
SETTINGS codes: int = 1024
```

An image ⇄ a coarser grid of codebook ids: an encoder, vector quantization, a decoder, trained to reconstruct.

`AudioCodec(CodecCmd[Codec[Temporal, Temporal[Seq]], [Temporal]])` Audio codec (EnCodec, residual quantization) new agnostic

```
(audio: Temporal, /) -> Codec[Temporal, Temporal[Seq]]
SETTINGS codebooks: int = 8, codes: int = 1024
```

A waveform ⇄ frames of codebook ids (a Temporal of Seqs): each codebook quantizes what the previous ones left (residual VQ).

`FitCodec(CodecCmd[None, [Codec[Any, Any], Dataset[Any]]])` Codec fitting (BPE merges, a vocabulary) new

```
(codec: Codec[Any, Any], data: Dataset[Any], /) -> None
```

agnostic

Fit a codec to a corpus by its own rule, in place: BPE merges the most frequent pairs until the vocabulary is full, a unigram model prunes. (A learned codec, VQ-VAE or EnCodec, is a model: train.)

`Batches(CodecCmd[Indexed[Any], ...])` Minibatches new agnostic

```
(*data: Container) -> Indexed[Any]
SETTINGS size: int, shuffle: bool = True
```

Containers in memory as a dataset: random minibatches of size examples, the containers sliced together (a tuple per batch when there are several). Indexed, so it can be curated and split.

`Mix(CodecCmd[Indexed[Any], ...])` Data mixture weights agnostic

```
(*datasets: Indexed[Any]) -> Indexed[Any]
SETTINGS weights: list[float]
```

Several datasets as one (web, code, books), each example weighted so that batches draw from them in proportion to the weights: the mixture as one Indexed, to curate and split like any.

`SaveLoadCmd[O, **R = []]` Model ⇄ file

```
(*R) -> O
```

A model's state to a file, and back.

`Save(SaveLoadCmd[str, [nn.Module, str]])` Save (a checkpoint) new agnostic

```
(model: nn.Module, path: str, /) -> str
```

Write a model's state (its state_dict: parameters and buffers) to a file, and return the path written.

`Load(SaveLoadCmd[None, [nn.Module, str]])` Load (a checkpoint) new agnostic

```
(model: nn.Module, path: str, /) -> None
```

Read a file Save wrote into a model of the same architecture, in place (load_state_dict).

`CurateCmd[C, **R = []]` Indexed → Subset

```
(data: Indexed[C], /, *R) -> Subset[C]
indices(data: Indexed[C], /, *R) -> Tensor
```

Keep, drop, reorder or resample examples, looking at all of them (blocking): a sub-dataset of the one given. A method only chooses the indices; the group makes them a Subset, which batches, iterates and logs them.

`Dedup(CurateCmd[Any])` Data deduplication agnostic

```
indices(data: Indexed[Any], /) -> Tensor
```

Drop exact copies across the whole dataset (by hash): the first of each kept.

`NearDedup(CurateCmd[Any])` Near-duplicate removal (MinHash / LSH) new agnostic

```
indices(data: Indexed[Any], /) -> Tensor
SETTINGS threshold: float = 0.8
```

Drop examples whose MinHash signatures collide with an earlier one's above a similarity threshold: dedup beyond exact copies.

KeepBest(CurateCmd[Any, [Callable[[Any], Value]]]) Data filtering by fraction new agnostic
`indices(data: Indexed[Any], score: Callable[[Any], Value], /) -> Tensor`
SETTINGS `keep: float = 0.5`
The best-scoring fraction of the whole dataset: every score first, then the ones above their quantile.

Curriculum(CurateCmd[Any, [Callable[[Any], Value]]]) Curriculum learning agnostic
`indices(data: Indexed[Any], difficulty: Callable[[Any], Value], /) -> Tensor`
The dataset sorted by difficulty, easy first: a global sort, so it reads every batch before the first comes out.

Subsample(CurateCmd[Any]) Random subsampling new agnostic
`indices(data: Indexed[Any], /) -> Tensor`
SETTINGS `fraction: float, seed: int = 0`
A random fraction of the examples, without replacement: a smaller run, a quick look.

Resample(CurateCmd[Any, [Callable[[Any], Value]]]) Resampling (importance sampling, bootstrap) new agnostic
`indices(data: Indexed[Any], weight: Callable[[Any], Value], /) -> Tensor`
SETTINGS `n: int | None = None, seed: int = 0`
n examples drawn with replacement, each in proportion to its weight: importance sampling, or the bootstrap when every weight is 1 (n defaults to the dataset's size).

Rebalance(CurateCmd[Any, [Callable[[Any], Tensor]]]) Class rebalancing (over- and undersampling) new agnostic
`indices(data: Indexed[Any], label: Callable[[Any], Tensor], /) -> Tensor`
SETTINGS `mode: str = "oversample", seed: int = 0`
As many examples of each class as of the largest ("oversample", repeating the rarer ones) or the smallest ("undersample", dropping some of the commoner ones).

PartitionCmd[C, **R = []] Indexed → splits of Subsets

```
(data: Indexed[C], /, *R) -> tuple[tuple[Subset[C], ...], ...]
parts(data: Indexed[C], /, *R) -> tuple[tuple[Tensor, ...], ...]
```

Split a dataset into sub-datasets: a list of splits, each a tuple of parts (train and test, or train, validation and test). A method only chooses the indices of each part; the group makes them Subsets.

TrainTestSplit(PartitionCmd[Any]) Train / validation / test split new agnostic
`parts(data: Indexed[Any], /) -> tuple[tuple[Tensor, ...], ...]`
SETTINGS `fractions: tuple[float, ...] = (0.8, 0.1, 0.1), label: Callable[[Any], Tensor] | None = None, seed: int = 0`
One split, at random, into parts of the given fractions (train, validation, test), each class in the same proportions in every part when label says what an example's class is (stratified).

Folds(PartitionCmd[Any]) Splits from a splitter new agnostic
`parts(data: Indexed[Any], /) -> tuple[tuple[Tensor, ...], ...]`
SETTINGS `splitter: Splitter, label: Callable[[Any], Tensor] | None = None, groups: Callable[[Any], Tensor] | None = None`
The splits of any scikit-learn splitter, each a (train, test) pair: label gives it the classes to stratify by, groups the groups to keep together (each called on batches, one value per example).

KFold(Folds) k-fold cross-validation new agnostic
→ `sklearn.model_selection.KFold, StratifiedKFold, GroupKFold, TimeSeriesSplit`
SETTINGS `splitter: Splitter, label: Callable[[Any], Tensor] | None = None, groups: Callable[[Any], Tensor] | None = None`
k (train, test) splits, each example tested once: plain, stratified by class, by group, or forward in time.

NetworkCmd[O, **R = []] ... → Model

```
(*R) -> O
```

Make a model, or set its weights directly: initialized, copied in, a frozen copy, a classical estimator.

FrozenCopy(NetworkCmd[nn.Module, [nn.Module]]) Frozen copy (reference / target network) agnostic
`[M: Module](model: M, /) -> M`
A copy that no longer trains: DPO's and RLHF's reference, DQN's target network.

Sklearn[E: Estimator](NetworkCmd[SkModel[E]]) scikit-learn estimator new agnostic
`() -> SkModel[E]`
SETTINGS `estimator: E`
Make a classical model from a scikit-learn estimator, or anything with its API (xgboost, lightgbm, umap): a fresh copy each call, so each is a model of its own. FitEstimator fits it; then its predict, predict_proba, transform, ... are expressions on batches.

Trees(Sklearn[Estimator]) Decision trees, random forests, gradient boosting new agnostic

→ sklearn.ensemble.HistGradientBoostingClassifier / Regressor, RandomForestClassifier, sklearn.tree.DecisionTreeClassifier, xgboost, lightgbm

SETTINGS estimator: E

Decision trees on feature vectors: one (CART), many on bootstrap samples averaged (a random forest), or each fitted to the errors of the ones before it (gradient boosting). Classifier or regressor, by the estimator.

SVM(Sklearn[Estimator]) Support vector machines (kernel SVM) new agnostic

→ sklearn.svm.SVC, SVR

SETTINGS estimator: E

The widest-margin separator in a kernel's feature space (a linear one is a linear layer with Hinge).

NaiveBayes(Sklearn[Estimator]) Naive Bayes new agnostic

→ sklearn.naive_bayes.GaussianNB, MultinomialNB

SETTINGS estimator: E

A generative classifier whose features are independent given the class: counts or one Gaussian per feature.

GaussianProcess(Sklearn[Estimator]) Gaussian processes new agnostic

→ sklearn.gaussian_process.GaussianProcessRegressor, GaussianProcessClassifier

SETTINGS estimator: E

A distribution over functions, set by a kernel: a prediction and its uncertainty, from every training example.

KMeans(Sklearn[Estimator]) k-means new agnostic

→ sklearn.cluster.KMeans, MiniBatchKMeans

SETTINGS estimator: E

k centroids, each example to its nearest and each centroid to the mean of its examples, until nothing moves; predict gives a new example's cluster.

GMM(Sklearn[Estimator]) Mixture models (GMM, EM) new agnostic

→ sklearn.mixture.GaussianMixture, BayesianGaussianMixture

SETTINGS estimator: E

k weighted Gaussians fitted by expectation–maximization: a density, and soft clusters by responsibility.

PCA(Sklearn[Estimator]) PCA (dimensionality reduction) new agnostic

→ sklearn.decomposition.PCA, IncrementalPCA, TruncatedSVD

SETTINGS estimator: E

A linear map onto the directions of most variance; transform projects onto them.

Standardize(Sklearn[Estimator]) Standardization (feature scaling) new agnostic

→ sklearn.preprocessing.StandardScaler, MinMaxScaler, RobustScaler

SETTINGS estimator: E

Each feature shifted and scaled (to mean 0 and variance 1, or into a range), by statistics of the training data only, so that the validation and test data are scaled as it was.

Transfer(NetworkCmd[None, [Weights, Weights]]) Pretrain → finetune (transfer) agnostic

(target: Weights, source: Weights, /) -> None

Start a model from a trained one: copy into the target the source's weights whose names and shapes match (the body), keep the target's head.

Init(NetworkCmd[None, [Weights]]) Initialization agnostic

(model: Weights, /) -> None

Fan-in normal for matrices, zeros for output projections: each residual block starts as the identity.

DeployCmd[X, **R = []] Model!

(x: X, /, *R) -> None

Change a model's nodes in place, one at a time: quantize them, shard them, freeze them, add an adapter.

Freeze(DeployCmd[nn.Module, [Callable[[str], bool]]]) Freezing agnostic

(model: nn.Module, trainable: Callable[[str], bool], /) -> None

Train only the parameters whose name passes a test.

LoRA(DeployCmd[nn.Module]) LoRA model

(model: nn.Module, /) -> None

SETTINGS rank: int = 8

W + B·A on chosen linear layers, the base frozen: the finetuning update, kept low-rank.

`Adapter(DeployCmd[nn.Module])` Adapters — model

```
(model: nn.Module, /) -> None
SETTINGS bottleneck: int = 8
```

After chosen frozen layers, a small bottleneck MLP that trains alone, starting at zero.

`PromptTuning(DeployCmd[nn.Module])` Prompt tuning — model

```
(model: nn.Module, /) -> None
SETTINGS n: int
```

Learned vectors in front of a frozen model's input; only they train.

`Quantize(DeployCmd[Weights])` Quantization (int8 / int4) — model

```
(model: Weights, /) -> None
SETTINGS bits: int = 8
```

Weights rounded to 2^(bits−1) − 1 levels a side per row: computing as the low-precision model would.

`LowPrecisionInference(DeployCmd[Weights])` Low precision: inference — model

```
→ Quantize
```

int8 / int4 weights.

`Shard(DeployCmd[nn.Module])` Parallelism — agnostic

```
→ torch.distributed
```

Data, sharded, tensor, pipeline, expert, sequence: fully_shard shards a module in place; DTensor places its weights.

`IndexCmd[O, **R = []]` … → Index | Index!

```
(*R) -> O
```

Make an index or keep one up to date: an empty one, its structure learned, insert, delete, add to a buffer.

`NewIndex(IndexCmd[Index, [Value]])` Index (an empty one) new — agnostic

```
(like: Value, /) -> Index
SETTINGS kind: str = "flat"
```

An empty index for vectors like these (their size): flat (exact), IVF (clusters), HNSW (a neighbour graph), PQ (compressed codes), LSH (hash buckets), inverted (BM25), sum tree (by priority). Statements train and fill it.

`TrainIndex(IndexCmd[None, [Index, Value]])` Index training (IVF centroids, PQ codebooks) new — agnostic

```
(index: Index, sample: Value, /) -> None
```

Learn the index's structure from a sample of the vectors, in place: k-means centroids for IVF, codebooks for PQ (flat and HNSW have nothing to learn). CREATE INDEX, before any rows.

`Insert(IndexCmd[None, [Index, Dataset[Set]]])` Index insertion new — agnostic

```
(index: Index, data: Dataset[Set], /) -> None
```

Add every batch of the dataset to the index, in place (FAISS add, INSERT). An index over an expression's output (embed(x)) goes stale when that expression's model is trained again: build it again.

`Delete(IndexCmd[None, [Index, Tensor]])` Index deletion new — agnostic

```
(index: Index, ids: Tensor, /) -> None
```

Remove elements from the index by id, in place (FAISS remove_ids, DELETE).

`Extend(IndexCmd[None, [Buffer[Container], Container]])` Continual learning (replay) — agnostic

```
[C: Container](buffer: Buffer[C], batch: C, /) -> None
```

Add a batch to a bounded buffer, in place, dropping the oldest (or the lowest priority): past experience to train on again (continual learning, off-policy RL, prioritized replay).

MISC · not placed in a group yet

`MiscOp` not placed yet

```
(*args: Any, **named: Any) -> Any
```

An op not placed in a group yet.

`Chinchilla(MiscOp)` Compute-optimal allocation (Chinchilla) — agnostic

```
(compute: float, /) -> tuple[float, float]
```

A compute budget → (parameters, tokens): C ≈ 6·N·D with D ≈ 20 tokens per parameter.

Blank(Start[Holes[Container], [Container]]) Blank (where generation starts) new agnostic
`[C: Container](like: C, /) -> Holes[C]`
Every element unknown: all [MASK] for classes, pure noise for real values.

LMTree(Start[PartialTree[Temporal, Tensor], [Predictor, Temporal]]) An LM as a tree agnostic
`(model: Predictor, prompt: Temporal, /) -> PartialTree[Temporal, Tensor]`
A node is a sequence (Holes: known up to its end), its options the next-token distribution, an edge appends.

EnvTree(Start[PartialTree[Any, Tensor], [Environment]]) MDP (an environment as a tree) agnostic
`(env: Environment, policy: Predictor | None = None, /) -> PartialTree[Any, Tensor]`
A node is a state, its options the policy's actions (uniform with none), an edge leads where the environment steps to, with a reward and whether it ended.

LeftToRight(Order) Autoregressive order new agnostic
`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`
STATE `Holes[C] — known up to i`
Fill the first hole: one element at a time, in order (the holes view of decoding; walk gives the tree view).

MostConfident(Order) Unmasking (MaskGIT, masked diffusion) new agnostic
`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`
STATE `Holes[C] — some holes at 1, the rest known`
Fill the holes the model is surest of, more each step (a cosine schedule), until none is left.

AllAtOnce(Order) Sampling a flow (diffusion) new agnostic
`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`
STATE `Holes[C] — every element at level t`
Lower every level by one step along the predicted velocity (Euler): noise, undone.

PerElement(Order) Diffusion forcing sampling new agnostic
`(holes: Holes[Any], pred: Prediction, /) -> Holes[Any]`
STATE `Holes[C] — a level for each element`
Each element follows its own schedule (the near future clean, the far future noisy): between LeftToRight and together.

MiscCmd not placed yet

`(*args: Any, **named: Any) -> Any`

A command not placed in a group yet.

ReplayBuffer(MiscCmd) Replay buffer (empty) new agnostic
`() -> Buffer[Any]`
SETTINGS `size: int, priority: bool = False`
An empty buffer of at most size examples, read by sampling (uniformly, or by priority); Extend fills it.

Intents · groups left open (resolve.py)

An intent is written where a method could be. A resolver picks one of its candidates (the default, a given choice, or by cost) and one rewrite lowers the program to concrete methods.

AugmentImage(Intent, ConvertOp[Grid, Grid])
CANDIDATES `RandomFlip(), RandomCrop(), ColorJitter(), GaussianNoise()`
Some label-preserving random change of an image: a flip, a crop, colour jitter, noise.

Decode(Intent, ChooseOp)
CANDIDATES `Greedy(), Sample(), Beam(k=4)`
Some way to walk an LM's tree: the likeliest token, a sample, a beam.

FillOrder(Intent, Order)
CANDIDATES `AllAtOnce(), MostConfident(), LeftToRight(), PerElement()`
Some order to fill holes in: all together (diffusion), the surest first, left to right, each its own level.

Mixer(Intent, LayerOp[Temporal, Temporal])
CANDIDATES `Layer(module=MultiheadAttention( (out_proj): NonDynamicallyQuantizableLinear(in_features=64, out_features=64, bias=True) )), SSM(d=64), Layer(module=GRU(64, 64, batch_first=True))`
Some way for a sequence's elements to read the earlier ones: attention, a selective scan, a recurrence.

`TrainLength(Intent, TrainCmd[nn.Module, [Callable[..., Tensor], Dataset[Any]]])`
CANDIDATES `Train(steps=100), Train(steps=200), Train(steps=400)`
Some amount of training: how long to update the network.

Meta-primitives and phenomena

Meta-primitives · properties of the groups

Bake in symmetry by sharing weights convert, layer
the same weights wherever the container relates elements: conv on a grid, attention on a set, messages on a graph

Let the input decide the computation convert, elementwise
attention weights, scan coefficients and expert choices computed from the data

Predict the hidden part from the visible part elementwise
every self-supervised signal is a split into Holes; generation fills them back

Turn one hard step into many easy ones elementwise, walk, extend
a token, a denoising step, an update at a time: a state, stepped

Learning is compression elementwise
bottlenecks, codebooks and students: the target, from a smaller description

Bootstrap from yourself train
targets made by your own (older) model: TD, self-distillation, self-play, AlphaZero

Amortize, or search elementwise, walk, extend, search
one forward pass that answers at once, or a tree grown by search

Every loss is a likelihood score
Loss()(pred, target): what the target holds picks which

Not every objective is a loss reduce, score
a return is a reduce of a path; its gradient comes from scoring the path, or there is none

Structure decides the computation convert, layer
the container picks what each element reads

Phenomena · observed, not computed

- **Implicit regularization of SGD**: a property of training dynamics, seen in eval curves
- **Double descent / benign overfitting**: seen by sweeping model or data size
- **Grokking**: delayed generalization in long runs
- **Superposition**: more features than dimensions; measured with sparse autoencoders
- **General methods + scale beat hand-engineering**: an empirical lesson (the bitter lesson), not an op